# Nvidia's Stock Returns Prediction Using Machine Learning Techniques for Time Series Forecasting Problem

*Justin Yuslih*[1], *Bryant Marchianno Jefferson*[2], *Aufa Dana Murtadha*[3], *Ivan Michael Lawrence Sanjaya*[4], *Yustinus Evan Kusuma*[5]

## Abstract

Forecasting stock return percentages remains a formidable challenge in financial time-series analysis due to the non-linear, noisy, and highly dynamic nature of stock markets. Most existing literature relies strictly on basic historical price data (OHLCV) without considering the broader economic environment, often yielding "black-box" models that lack interpretability for institutional investors. To address these limitations, this study proposes a comprehensive machine learning framework to predict the daily stock return percentages of NVIDIA Corporation (NVDA) utilizing an extensive dataset spanning from 1999 to 2026. The proposed framework enriches traditional market data with 71 advanced engineered features, including market volatility indexes (VIX), semiconductor ETF (SMH) returns, corporate earnings-event variables, and relative strength indicators against the SPY and QQQ benchmarks. We rigorously evaluate traditional machine learning models (Random Forest, XGBoost, LightGBM, Ridge Regression, and Support Vector Regression) alongside a deep learning Bidirectional LSTM with an Attention mechanism (BiLSTM-Attention) and a Stacking Meta-Learner ensemble framework. To simulate realistic trading scenarios and mitigate cumulative drift, a rolling one-step forecasting evaluation strategy is implemented. Furthermore, Explainable Artificial Intelligence (XAI) techniques leveraging SHAP (SHapley Additive exPlanations) are incorporated to deconstruct the models' decision-making processes. The empirical results demonstrate that the BiLSTM-Attention model delivers superior performance in capturing directional trends, achieving a Directional Accuracy of 54.85% overall and up to 58.7% during a 3-month rolling window. Concurrently, the LightGBM and Stacking Ensemble frameworks exhibit robust nominal price tracking stability, registering a highly competitive MAPE of 1.86% under volatile market conditions. SHAP analysis successfully reveals that short-term volatility indicators (rolling standard deviation) and price band thresholds (Bollinger Bands) serve as the most dominant features driving the prediction outputs. This study provides significant contributions to the development of robust, transparent, and legally interpretable quantitative trading architectures for high-growth technology equities.

## Keywords

machine learning | nvidia | stock return forecasting | bilstm attention | stacking ensemble | shap

## 1. Introduction

Since stock prices are affected by nonlinear, noisy, and time-dependent market dynamics; therefore, financial stock return forecasting remains a challenging research problem. Previous studies have shown that machine learning and deep learning models are increasingly used to capture complex patterns in financial time series, although predictive performance is highly dependent on feature design, validation strategy, and market context (Sezer et al., 2020; Kumbure et al., 2022). In the case of Nvidia Corporation, the prediction task is particularly relevant because its valuation is connected not only to its own historical price behavior, but also to broader technology, semiconductor, artificial intelligence, gaming, and cryptocurrency-related market conditions (Chlebus et al., 2021).

Existing literature has explored several approaches for stock market forecasting. Chlebus et al. (2021) developed machine learning models for Nvidia stock returns using technical, fundamental, behavioral, and market-related exogenous variables. Other studies have emphasized the usefulness of technical indicators such as moving averages, MACD, RSI, and volatility-based features in deep learning models, particularly LSTM-based architectures (Phuoc et

al., 2024; Nabipour et al., 2020). More recent research also suggests that hybrid and optimized deep learning methods, including LSTM-ARO and BiLSTM variants, can improve forecasting performance in stock price prediction tasks (Gülmez, 2023; Sang & Li, 2024).

Furthermore, several studies highlight that feature engineering plays a critical role in financial forecasting performance. Market volatility indicators, lag-based momentum variables, and contextual market proxies are often more influential than model complexity alone. Kumbure et al. (2022) emphasized that the predictive capability of machine learning models in financial forecasting is strongly dependent on the relevance and quality of engineered features. Similarly, Sezer et al. (2020) reported that technical indicators and sequential temporal representations significantly improve the ability of deep learning models to identify hidden dependencies within stock market data.

Despite the growing number of machine learning applications in financial forecasting, several limitations remain in existing studies. First, many studies rely primarily on historical OHLCV data without incorporating broader contextual financial variables such as market volatility proxies, sector-level indicators, or earnings-related information. Second, several studies evaluate prediction performance only using conventional train-test approaches, which may introduce optimistic results in time-series environments. Third, explainability analysis is often limited, making it difficult to understand which features contribute most significantly to prediction behavior.

To address these limitations, this study proposes a comprehensive machine learning framework for predicting Nvidia (NVDA) stock return percentages using a combination of technical indicators, volatility-based variables, market context features, and explainable artificial intelligence (XAI) analysis. The proposed framework integrates traditional machine learning methods, including Random Forest, XGBoost, LightGBM, Ridge Regression, and Support Vector Regression (SVR), together with a deep learning BiLSTM-Attention architecture and a Stacking Meta-Learner ensemble approach.

In addition to historical NVDA stock data, this research incorporates contextual financial features such as VIX volatility indicators, semiconductor ETF (SMH) returns, earnings-event variables, relative strength indicators against SPY and QQQ, rolling volatility measures, and momentum-based features. The objective is to construct a richer representation of market behavior capable of improving prediction robustness under different market conditions.

This study also applies rolling one-step forecasting evaluation to simulate realistic stock market prediction scenarios. Unlike conventional recursive forecasting methods, rolling evaluation continuously uses the previous actual closing price when generating future predictions, thereby reducing cumulative forecasting drift and improving evaluation reliability.

Moreover, Explainable Artificial Intelligence (XAI) techniques using SHAP (SHapley Additive exPlanations) are incorporated to interpret model behavior and identify the most influential variables contributing to prediction outputs. This explainability analysis is important because financial forecasting models are often criticized as “black-box” systems that lack interpretability for investors and analysts.

The main objective of this research is to evaluate the effectiveness of machine learning and deep learning approaches in predicting NVIDIA stock return percentages using extensive feature engineering and rolling forecasting validation. This study also aims to identify the most influential financial indicators contributing to prediction performance and to analyze whether contextual market features improve forecasting robustness.

The contribution of this research lies in the integration of updated NVIDIA market data from 1999 to 2026, contextual financial indicators, rolling forecasting evaluation, and SHAP-based explainability analysis into a unified forecasting framework.

## 2. Literature Review

Financial stock forecasting has long been considered a difficult research problem because stock markets are highly dynamic, nonlinear,

noisy, and influenced by many external factors including investor psychology, macroeconomic conditions, geopolitical events, and market sentiment. Traditional statistical forecasting methods such as ARIMA, linear regression, and GARCH models have historically been used to model financial time series. However, these approaches often rely on assumptions such as stationarity and linearity, which may not accurately represent modern stock market behavior, particularly during periods of high volatility and rapid market regime changes.

As computational power and financial data availability increased, machine learning approaches became increasingly popular for stock market prediction tasks. Machine learning methods are capable of identifying nonlinear relationships and hidden interactions between financial variables without relying heavily on strict statistical assumptions. Because of this flexibility, machine learning models are often better suited for modeling complex financial systems compared to traditional econometric methods.

Nabipour et al. (2020) compared multiple machine learning algorithms including Decision Trees, Random Forest, Support Vector Machines, and Artificial Neural Networks for stock market forecasting. Their study demonstrated that nonlinear machine learning models consistently outperform simpler linear approaches in capturing stock market behavior. The research also highlighted that preprocessing quality and feature engineering significantly influence forecasting performance.

Among machine learning approaches, ensemble learning methods such as Random Forest, XGBoost, and LightGBM have gained substantial attention because of their strong performance on structured financial datasets. Random Forest utilizes multiple decision trees to reduce overfitting and improve generalization capability. XGBoost and LightGBM extend this idea further through gradient boosting optimization, allowing models to learn complex nonlinear interactions between features more efficiently.

Several studies reported that boosting-based algorithms achieve highly competitive performance in financial forecasting tasks. XGBoost is particularly effective because of its regularization capability, while LightGBM is known for its computational efficiency and ability to handle large feature spaces. These characteristics make boosting algorithms highly suitable for stock return prediction problems involving numerous technical indicators and contextual market variables.

Support Vector Regression (SVR) is another widely used machine learning method in financial forecasting research. SVR utilizes kernel-based learning to model nonlinear relationships within noisy datasets. Previous studies frequently reported that SVR performs competitively in stock return forecasting because financial data often contains complex nonlinear structures and outlier behavior.

In recent years, deep learning architectures have become increasingly popular in financial forecasting research due to their ability to automatically learn sequential temporal representations from time-series data. Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) architectures are especially important because they are specifically designed to process sequential information and preserve temporal dependencies over long periods.

Sezer et al. (2020) explained that deep learning methods are capable of extracting hidden hierarchical patterns from financial time series data without requiring extensive manual feature interaction engineering. Their review demonstrated that LSTM-based models consistently achieve strong forecasting capability in stock market prediction tasks due to their ability to capture long-term temporal dependencies.

LSTM architectures solve several limitations found in traditional RNN models, particularly the vanishing gradient problem that occurs during long sequence training. Because stock prices often depend on historical movement patterns across multiple time windows, LSTM networks are highly suitable for financial forecasting applications.

More advanced sequence-learning approaches such as Bidirectional LSTM (BiLSTM) have also been introduced to improve

temporal representation learning. Unlike conventional LSTM models that process sequences in a single direction, BiLSTM architectures process temporal information in both forward and backward directions. This allows the model to learn richer contextual dependencies within financial time series.

Gülmez (2023) proposed optimized LSTM architectures for stock market prediction and demonstrated that proper optimization strategies significantly improve forecasting stability. Similarly, Sang and Li (2024) introduced hybrid BiLSTM-based forecasting methods that achieved stronger prediction performance compared to standard LSTM models in several financial forecasting scenarios.

Attention mechanisms have also become increasingly important in deep learning-based financial forecasting systems. Attention layers allow neural networks to focus selectively on the most relevant temporal information during prediction generation. In financial markets, certain historical periods may contain more important signals than others, especially during high-volatility events, earnings announcements, or sudden trend reversals. Therefore, integrating attention mechanisms into BiLSTM architectures may improve prediction interpretability and forecasting performance.

In addition to model architecture, feature engineering remains one of the most critical factors affecting stock market prediction performance. Kumbure et al. (2022) conducted a systematic review involving more than one hundred machine learning studies and concluded that feature quality often contributes more significantly to forecasting performance than model complexity alone.

Technical indicators such as Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), Bollinger Bands, moving averages, and volatility measures are among the most widely used features in stock forecasting systems. These indicators are designed to capture momentum behavior, volatility conditions, price trends, and potential reversal patterns.

Several studies also emphasize the importance of incorporating contextual market variables into financial forecasting systems. Variables such as volatility indexes (VIX), sector ETFs, macroeconomic indicators, and earnings-event information help models capture broader market behavior that cannot be fully represented by historical stock prices alone.

Chlebus et al. (2021) developed one of the most relevant studies regarding NVIDIA stock return forecasting. Their research integrated technical indicators, cryptocurrency-related variables, Google Trends data, and gaming-related information into multiple machine learning forecasting models. The study demonstrated that contextual market variables improve predictive capability, although forecasting performance remained sensitive to market volatility conditions.

Despite significant progress in financial forecasting research, several limitations still remain in existing literature. First, many studies rely heavily on historical OHLCV data while excluding broader contextual market indicators. Second, explainability analysis is frequently limited, causing machine learning forecasting systems to behave as “black-box” models with limited interpretability. Third, some studies use random train-test splitting approaches that may unintentionally introduce future information leakage in time-series datasets.

Another important limitation is that many studies evaluate forecasting performance only using return-based metrics without reconstructing rolling predicted stock prices. In real-world financial environments, rolling price evaluation is important because it better reflects how forecasting systems behave during continuous market operation.

To address these limitations, this study proposes a comprehensive NVIDIA stock forecasting framework that integrates contextual market variables, advanced feature engineering, rolling one-step forecasting evaluation, multiple machine learning and deep learning architectures, and SHAP-based Explainable Artificial Intelligence analysis.

| Aspect | CEEj 2021 | This Notebook | Implications |
|---|---|---|---|
| Dataset | NVDA daily returns, 01/2012-12/2018; test 07/2018-12/2018 | NVDA yfinance 1999-2026; 6,529 rows ready to use | Notebook wins in latest time coverage |
| Initial features | >350 independent variables: technical, fundamental, behavioural, Nvidia-market proxies | 71 features: technical, VIX/SMH, IV proxy, earnings, lag/momentum, volume flow, relative strength | CEEj is broader; notebooks are more focused and easier to replicate |
| Model | SVR, KNN, XGBoost, LGBM, LSTM, ARIMA/ARIMAX, ranking ensemble | RF, XGBoost, LightGBM, BiLSTM-Attention, Ridge, SVR, Stacking | Notebook adds BiLSTM-Attention and XAI |
| Best result | SVR stationary best: RMSE .036014, MAE .024916; stationary ensemble RMSE .036106 | BiLSTM DA 54.85%; XGBoost/RF RMSE .03172; LightGBM rolling price MAE $2.05 | Similar findings: ensembles don't automatically win |

**Fig. 1**. Comparison table between previous research and our notebook

# 3.Materials and Methods

This study proposes a comprehensive machine learning framework for forecasting NVIDIA Corporation (NVDA) stock return percentages using historical financial data, contextual market indicators, extensive feature engineering, rolling one-step forecasting evaluation, and explainable artificial intelligence (XAI) analysis. The proposed methodology was developed to address several major limitations commonly found in stock market forecasting research, particularly issues related to data leakage, insufficient contextual representation, limited interpretability, and unrealistic evaluation procedures.

Financial time-series forecasting is considered one of the most difficult predictive modeling problems because stock prices are influenced by numerous interconnected factors including macroeconomic conditions, investor sentiment, geopolitical events, technological developments, market volatility, and sector-specific dynamics. Unlike conventional regression problems, stock market data exhibits high levels of noise, non-stationarity, volatility clustering, and abrupt structural changes. Consequently, forecasting systems require not only predictive capability but also robustness against unstable market conditions and changing financial environments.

Many conventional stock forecasting studies rely exclusively on historical OHLCV variables consisting of Open, High, Low, Close, and Volume data. However, relying solely on historical prices may limit the ability of machine learning models to capture broader market conditions affecting semiconductor companies such as NVIDIA. Therefore, this study extends the forecasting framework by incorporating contextual variables including volatility indexes, technology-sector benchmarks, semiconductor exchange-traded funds (ETFs), and engineered relative-strength indicators.

The overall forecasting pipeline was implemented using Python programming language and multiple machine learning libraries including Scikit-learn, TensorFlow, XGBoost, LightGBM, SHAP, and Optuna. The workflow consists of several stages including data collection, preprocessing, feature engineering, feature selection, hyperparameter optimization, model training, rolling forecasting evaluation, and explainability analysis.

To preserve forecasting realism and avoid future information leakage, chronological order was strictly maintained throughout all stages of training, validation, and testing. Unlike random train-test splitting approaches frequently used in conventional machine learning tasks, this study adopts time-based splitting and rolling one-step forecasting evaluation procedures. This methodology better represents real-world financial forecasting environments where future information is unavailable during training.

In addition, this research evaluates multiple machine learning and deep learning architectures to compare forecasting performance under different modeling assumptions. The evaluated models include Random Forest, XGBoost, LightGBM, Ridge Regression, Support Vector Regression (SVR), BiLSTM-Attention, and stacking ensemble architectures. Explainability analysis using SHAP (SHapley Additive exPlanations) was also incorporated to improve interpretability and identify the most influential variables contributing to prediction behavior.

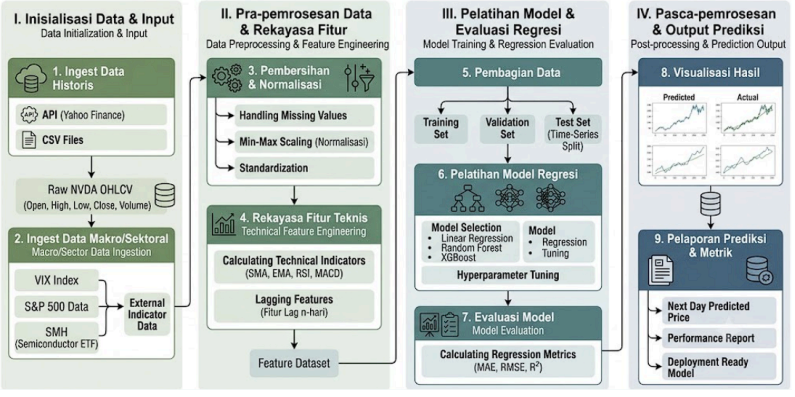



**Fig. 2**. Research Piepline

### 3.1 Dataset

The foundation of this quantitative framework rests upon a dense, multi-source dataset capturing the market dynamics of NVIDIA Corporation alongside key macroeconomic and sectoral benchmarks. The primary equity data was programmatically harvested from the Yahoo Finance API using the yfinance Python library. The core raw observation period established for this research spans from January 1999 through early 2026, encompassing approximately 6,870 raw trading days. This expansive temporal window provides an exceptionally robust historical baseline, exposing the models to diverse macroeconomic regimes, including the 2000 dot-com crash, the 2008 global financial crisis, the post-pandemic inflation surge, and the rapid expansion of the global artificial intelligence economy.

In financial forecasting, directly predicting raw closing prices ($p_t$) is statistically problematic due to non-stationarity, compounding scale effects, and the presence of unit roots, which violate the foundational assumptions of many machine learning algorithms. To resolve this, the target variable in this study is defined as the next-day stock return percentage, formalized as Target_Return_Besok. This transformation stabilizes the mean and variance of the distribution, ensuring scale invariance across decades of long-term capital appreciation. The mathematical formulation for the target variable at time t is expressed as:

$$\text{Target_Return_Besok}_t = \left( \frac{P_{t+1} - P_t}{P_t} \right) \times 100$$

Where $p_t$ and $p_{t+1}$ denote the adjusted closing prices of NVIDIA stock on day t and day t + 1, respectively. To prevent the predictive models from operating in a univariate vacuum, the feature space was enriched with four strategic exogenous market indicators that capture systematic risk, tech-sector velocity, and semiconductor industry health:

- CBOE Volatility Index (VIX): Utilized as a proxy for aggregate market fear, implied volatility, and macroeconomic uncertainty, which directly impacts investor risk appetite for high-beta tech equities.
- SPDR S&P 500 ETF Trust (SPY): Captures the systematic baseline, liquidity, and directional momentum of the broader United States equity market.
- Invesco QQQ Trust (QQQ): Monitors the performance of the Nasdaq-100 index, serving as the primary benchmark for large-cap growth and technology-driven corporate valuations.
- VanEck Semiconductor ETF (SMH): Tracks the MVIS US Listed Semiconductor 25 Index, providing an essential contextual proxy for industry-specific supply-demand metrics, hardware cycles, and manufacturing momentum directly tied to NVIDIA's core business.

Following raw data consolidation, extensive mathematical transformations were applied to derive complex multi-temporal technical indicators, rolling volatility metrics, historical lag sequences, and buying-selling pressure features. After filtering out observations affected by indicator initialization windows, the finalized dataset contains exactly 6,529 full-featured trading observations across 71 continuous predictive variables, presenting a high-dimensional regression landscape for the machine learning pipeline.

| Source | Feature Example | Function |
|---|---|---|
| NVDA OHLCV | Open, High, Low, Close, Volume | basis return, price level, and technical indicators |
| VIX + SMH | VIX_Close, VIX_spike, SMH_Return | risk regime and semiconductor context |
| IV proxy | HV_10, HV_20, HV_30, Vol_term_slope | historical volatility and risk slope |
| Earnings | is_earnings_day, days_to_earnings | event timing that can trigger volatility |
| Relative strength | alpha_vs_SPY, alpha_vs_QQQ, beta_vs_SPY | NVDA's position relative to the broader market |

**Fig. 3**. Feature Engineering Table

### 3.2. Data Collection and Preprocessing (wip)

The data collection process was performed using Yahoo Finance APIs integrated through Python-based financial data libraries. Historical stock prices and contextual market indicators were downloaded separately and subsequently merged into a unified dataset based on trading dates.

The preprocessing stage is considered extremely important in financial forecasting research because stock market datasets frequently

contain inconsistencies, missing values, rolling-window artifacts, and potential future information leakage. Consequently, preprocessing procedures were conducted carefully to preserve temporal integrity and ensure reliable forecasting evaluation.

### 3.2.1 Date Alignment and Synchronization

The process of date alignment and synchronization is a highly critical phase in the preprocessing of multivariate financial time-series data. Because the forecasting framework proposed in this study integrates data from multiple distinct market instruments, namely NVIDIA stock (NVDA), the implied volatility index (VIX), and three sectoral and macroeconomic Exchange-Traded Funds (SPY, QQQ, SMH) structural inconsistencies in trading calendars frequently occur. These calendar discrepancies can be caused by specific market holidays, temporary trading suspensions on certain assets, or historical differences in instrument inception dates (e.g., the historical track record of SMH begins later than the early foundational data of NVDA).

To overcome these structural challenges, a row-based relational inner join procedure was executed using the trading timestamp as the unique index key. Through this inner join approach, the data cleaning algorithm automatically eliminates date rows that lack complete observations across all instruments. For instance, if on a specific date the VIX data was not published but the NVDA equity market remained open, that observation row is entirely removed from the matrix. This alignment step is meticulously designed to ensure that every row in the feature space $x_t$ represents a globally valid and synchronous snapshot of market conditions on day t, thereby guaranteeing that no time-shift artifacts corrupt the causal representation between variables.

### 3.2.2 Missing Value Handling

In quantitative analysis, the handling of missing values (NaN) requires extreme caution because it directly impacts the convergence stability of gradient-based models and neural networks. In this study, the occurrence of NaN values is not caused by data extraction omissions, but rather is a mathematical consequence of the feature engineering process. Technical indicators such as the 50 day Simple Moving Average (SMA) or the 20 day Exponential Moving Average (EMA) inherently require a long lookback window, which causes the initial rows at the beginning of the dataset's timeline (Year 1999) to lack valid calculated values.

Many conventional studies apply imputation techniques such as mean imputation, forward-filling, or linear interpolation to preserve sample size. However, in the domain of stock return forecasting, such techniques are strictly avoided because they introduce artificial noise, incorrectly smooth out natural variance, and distort the heavy-tailed distributions characteristic of financial data.

Consequently, this study applies a complete truncation strategy. Any observation row containing at least one NaN value in any engineered column is permanently removed from the data matrix. This rigorous cleaning procedure is the primary reason the dataset was reduced from its initial raw size of approximately 6,870 trading days to exactly 6,529 clean, continuous observations devoid of information gaps.

### 3.2.3 Leakage Prevention

Data leakage (or lookahead bias) is one of the most common causes of practical deployment failures for machine learning models in financial markets. Leakage occurs when future information (t+k) inadvertently bleeds into the training set or feature engineering process at the present time (t), resulting in artificially superior out-of-sample evaluation performance that collapses entirely during live trading deployment. To fortify the empirical validity of this framework, data leakage prevention was structurally hardcoded into the pipeline through four layers of protection:

- Target Variable Isolation, the target variable Target_Return_Besok was constructed exclusively by chronologically shifting the next day's closing price ($p_{t+1}$) backward, ensuring it is never present or extractable

within the explanatory feature matrix ($x_t$).

- Causal Continuous Features, all technical indicators, rolling volatility metrics, and movement statistics (such as moving averages or rolling standard deviations) are formulated using closed past windows ([t-k, t]). The calculation window strictly terminates at the closing price of day t, ensuring no intra-day information from tomorrow contaminates today's features.
- Scalar Parameter Separation, data normalization procedures via numerical scaling are not applied to the entire global dataset simultaneously. Statistical estimation parameters, such as the median and interquartile range (IQR), are learned exclusively from the training partition. The multiplier parameters from this training data are subsequently utilized to transform the validation and testing datasets. This approach ensures that the variance or the upper/lower bounds of future data in the test set remain entirely unknown to the model during the training phase.
- Chronological Time-Series Evaluation, the use of random k-fold cross-validation algorithms is strictly prohibited in this study. Such methods randomly assign past data as prediction targets based on future data, fundamentally violating the law of time causality in finance.

### 3.2.4 Data Transformation and Scaling

Raw data extracted via financial APIs often arrive in complex multi-index dataframe formats, where columns are hierarchically separated by asset name and price metrics (e.g., a high-level separation between Close and Volume columns for all five instruments). To accelerate computational execution on numerical arrays and simplify variable manipulation, this hierarchical structure was flattened into standardized, single-level column string identifiers, such as NVDA_Close, VIX_Close, SMH_Volume, and Return_Hari_Ini.

Diagnostic inspections for data anomalies were conducted using visual boxplots and the Isolation Forest algorithm. The analysis revealed the presence of extreme price spikes and deep drawdowns occurring during specific global crisis periods. Unlike non-financial tabular data processing, where outliers are frequently deleted or winsorized, this study deliberately preserves all volatility outliers completely intact. In quantitative finance, extreme volatility (black-swan events) carries highly valuable structural signals regarding market regime shifts. Removing these outliers would severely degrade the models' capacity to recognize tail risks and diminish their robustness when facing real-world market turbulence.

However, the extremely wide variation in raw value scales across variables for example, trading volumes reaching millions of units compared to return percentages in small single digits can cause convergence failures in parametric models like Ridge Regression, SVR, and BiLSTM-Attention networks. These models are highly sensitive to feature magnitude. If standard scaling (such as MinMax Scaler or Standard Z-Score Scaler) is used, the preserved financial outliers would heavily distort the data distribution, compressing normal variance into a very narrow band and triggering the vanishing gradient phenomenon. To resolve this, the study implements the robust RobustScaler algorithm. The mathematical transformation of the RobustScaler is formulated as follows:

$$X_{\text{scaled}} = \frac{X - \text{median}(X)}{\text{IQR}(X)}$$

Where IQR(X) is the Interquartile Range, calculated as the difference between the Third Quartile ($Q_3$) and the First Quartile ($Q_1$). This median and IQR based approach guarantees that data centering and feature distribution remain stable, unaffected by extreme deviations, and capable of maintaining stable gradient flows during the backpropagation process in deep learning architectures.

## 3.3. Feature Engineering

The feature engineering procedure in this framework is designed to extract hidden non-linear patterns, short-to-medium-term

memory persistence, and liquidity dynamics embedded within the raw market data. By transforming basic closing price metrics into information-rich derivative indicator variables, the models are provided with a higher structural capacity to separate alpha signals from dominant market noise. This comprehensive feature engineering process resulted in a total of 71 continuous predictive variables grouped into five primary thematic blocks.

### 3.3.1 Technical Indicators

Technical indicators act as mathematical filters that convert daily closing price fluctuations into continuous signals representing market saturation, momentum trends, and structural reversals. This study integrates the high-level pandas_ta library to generate key technical metrics using industry-standard lookback parameters:

- Relative Strength Index (RSI) is a bounded momentum oscillator measuring the speed and magnitude of internal price changes over a 14 day window. RSI values are constrained between a scale of 0 and 100 to identify overbought conditions when values reach 70 or above, and oversold conditions when values drop to 30 or below.
- Moving Average Convergence Divergence (MACD) captures macroeconomic trend shifts by calculating the difference between the 12 day and 26 day Exponential Moving Averages (EMA). This difference is mathematically paired with a 9 day EMA signal line to accurately detect momentum trend crossovers.
- Simple and Exponential Moving Averages (SMA/EMA) are calculated across multiple time horizons covering short-term 7 day, medium-term 20 day, and long-term 50 day periods. These moving averages serve as dynamic anchor levels tracking the macro support and resistance lines of NVIDIA's price movement.
- Average True Range (ATR) serves as a pure price-based volatility metric measuring the maximum movement range from the absolute values of today's high-low difference, today's high against yesterday's close, and today's low against yesterday's close over a 14 day lookback window.
- Bollinger Bands represent a geometric price envelope construction consisting of a 20 day SMA centerline flanked by an upper band and a lower band set at two standard deviations. Specific features extracted from this architecture provide the models with direct spatial information regarding phases of volatility expansion or price compression.

### 3.3.2 Volatility Features

Volatility in financial markets exhibits a unique property known as volatility clustering, where periods of high volatility tend to be followed by high volatility, and vice versa. To capture this autoregressive risk property, the feature engineering framework synthesizes robust rolling risk metrics:

- Rolling Standard Deviation tracks the standard deviation of NVIDIA's closing price over a short-term trailing window of one trading week, providing an instantaneous quantitative response to sudden price shocks.
- Historical Volatility (HV) is formulated across three strictly structured rolling time horizons encompassing 10 day, 20 day, and 30 day windows. This feature measures the annualized standard deviation of logarithmic asset returns to enable the algorithms to identify structural shifts in macro risk regimes.
- Volatility Slope calculates the first derivative or the rate of change of short-term historical volatility against long-term volatility, providing a clear mathematical signal on whether the market is moving toward a stabilization phase or escalating into panic.

### 3.3.3 Relative Strength Features

NVIDIA stock does not move in isolation, as its valuation is heavily tied to semiconductor supply chains and global technology sector sentiment. Therefore, relative strength features are designed to model cross-asset interactions. By

calculating mathematical ratios, return spreads, and rolling covariances between NVIDIA's daily performance against macro external instruments namely SPY (S&P 500), QQQ (Nasdaq-100), and SMH (Semiconductor ETF) this framework explicitly isolates NVIDIA's specific movement behavior. These features help the models distinguish whether NVIDIA's price appreciation is driven by the company's idiosyncratic alpha generation or merely mirroring the systematic beta exposure of the broader equity market.

### 3.3.4 Lag Variables and Momentum Features

Given the time-series dependency of financial data on historical conditions, past internal states act as exceptionally strong predictive anchors. The feature engineering library builds an extensive lag structure to capture these temporal dependencies:

- Price Lags such as the variables lag_close_3 and lag_close_7 directly transmit the absolute closing price positions of NVIDIA from three and seven trading days ago into every model decision row.
- Momentum Trackers including the Return_Hari_Ini feature capture the daily percentage price change velocity calculated just before the market closes on the current day. This variable serves as the primary foundational stepping stone for the model to project momentum shifts into the following day.

### 3.3.5 Volume Flow Indicators

Stock price movements driven by heavy trading volume reflect strong institutional participation and trend sustainability, whereas light-volume price movements are generally considered market noise susceptible to sudden reversals. Therefore, volume-based liquidity flow indicators are fully integrated into the feature space:

- On-Balance Volume (OBV) acts as a running cumulative indicator that adds trading volume on days when prices rise and proportionately subtracts volume on days when prices fall, effectively mapping historical capital accumulation or distribution pressure.
- Money Flow Index (MFI) serves as an enhanced volume-weighted momentum oscillator. Unlike RSI which evaluates only the price component, MFI integrates trading volume into the 14 day calculation window to provide a mathematical visualization of capital flow intensity entering or exiting the NVIDIA ecosystem.
- Chaikin Money Flow (CMF) measures excess liquidity accumulation by calculating the volume-weighted closing price position relative to the high-low price range over a 21 day cycle. Through this specific metric, institutional liquidity shifts become highly visible to the learning algorithms.

## 3.4. Data Preparation and Time-Based Split

To objectively evaluate the reliability and out-of-sample accuracy of all predictive models, the clean dataset containing 6,529 observations must be divided into separate partitions. As previously emphasized, uniform random train-test splitting methods are strictly prohibited as they destroy the chronological time dependency of time-series data and trigger severe lookahead bias. As a solution, this study implements a strict chronological time-based splitting strategy with an allocation ratio of 70%, 15%, and 15%. Specifically, the dataset division yields the following observation volumes:

- Training Set comprises the first 70% of the data chronologically, encompassing exactly 4,570 continuous trading observations. This set serves as the foundational optimization space where algorithms adjust internal weights, minimize loss functions, and map the mathematical decision structures from features to targets.
- Validation Set encompasses the subsequent 15% chronological data block consisting of 979 daily trading observations. This validation set is entirely isolated from the foundational weight-updating phase and is allocated exclusively for hyperparameter tuning via the Bayesian framework, tree

pruning execution, and the enforcement of early stopping mechanisms on neural networks.

- Testing Set secures the final 15% chronological data block containing exactly 980 daily trading observations. This testing set acts as a pure out-of-sample territory that is never seen by the models during the entire design, feature engineering, or parameter optimization cycles.

To convert the model output predictions back into nominal currency price values understandable in asset management operations, a rolling one-step ahead price reconstruction framework is implemented. Compared to recursive multi-horizon forecasting systems which are prone to exponential computational error accumulation, this one-step ahead framework consistently links the model projection directly to the actual closing price recorded on the previous day. The mathematical equation for reconstructing the nominal estimated price is formulated as follows:

$$\hat{P}_t = P_{t-1} \times \left(1 + \frac{\hat{R}_t}{100}\right)$$

Where $p_{t-1}$ represents the unmutated actual closing price on the previous trading day t-1, and $\widehat{Rt}$ denotes the model's predicted stock return percentage for the current day $t$. This rolling evaluation method precisely simulates a real-world systematic trading infrastructure.

### 3.5. Machine Learning and Deep Learning Models

To explore and map the high-dimensional landscape of 71 quantitative features against NVIDIA's stock return percentages, this study deploys seven modeling architectures with complementary theoretical characteristics. The comparative evaluation encompasses parametric linear shrinkage models, non-parametric bootstrap-based tree ensembles, highly scalable gradient boosting architectures, maximum-margin kernel algorithms, and deep sequential neural networks equipped with dynamic attention gates.

#### 3.5.1 Random Forest

The Random Forest Regressor algorithm formulated by Leo Breiman (2001) is a non-parametric ensemble learning method operating on the principle of bootstrap aggregation. In highly noisy financial time-series modeling, a single decision tree is exceptionally prone to severe overfitting due to its tendency to grow overly deep branches to memorize data outliers. Random Forest drastically reduces this vulnerability by injecting two structural randomization layers during the training phase.

The first randomization layer is executed by building each decision tree within the ensemble using independent bootstrap samples drawn randomly with replacement from the main training partition. The second randomization layer is applied at the internal node level, where the algorithm restricts the search space for the best feature to a random subset. The final prediction output of the Random Forest architecture is obtained through an enclosed averaging operation of the outputs from all independent trees within its ecosystem:

$$\hat{y}_i = \frac{1}{B}\sum_{b=1}^{B} T_b(x_i)$$

Where B represents the total number of decision trees in the ensemble, and $T_b(x_i)$ denotes the scalar prediction output from the b-th tree. Critical hyperparameters such as the number of estimators, maximum tree depth, and the minimum samples required for a node split were strictly optimized using the Optuna Bayesian framework.

#### 3.5.2 XGBoost

Extreme Gradient Boosting (XGBoost) developed by Tianqi Chen and Carlos Guestrin (2016) is a highly scalable implementation system of Gradient Boosted Decision Trees. Unlike bagging

methods that train models in parallel without interaction, XGBoost operates on a sequential boosting principle where each new tree is grown consecutively with a singular focus on correcting the residual errors left by the combination of prior trees. At the t-th boosting iteration, the objective function to be minimized is calculated as follows:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

The regularization term $\Omega(f_t)$ acts as an intrinsic penalty suppressing tree depth to prevent overfitting:

$$\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

To accelerate the optimization process of this objective function without losing mathematical accuracy, XGBoost adopts a second-order Taylor expansion approximation. Using the Optuna automated optimization library, crucial parameters such as the learning rate, maximum branching depth, and row subsample ratios were calibrated to suppress the validation error.

### 3.5.3 LightGBM

The Light Gradient Boosting Machine (LightGBM) is a gradient boosting framework specifically designed to optimize computational speed and memory efficiency on large-scale datasets. While conventional GBDT models build decision trees using a level-wise tree growth strategy, LightGBM adopts a leaf-wise tree growth strategy.

In the leaf-wise strategy, LightGBM selects the single specific leaf node possessing the maximum structural loss reduction value across the entire tree to split first. This produces asymmetrical, deeper, and highly specialized tree structures that are empirically proven to capture highly complex non-linear interactions. To mitigate the inherent high variance risk of these deep-leaf trees, the objective function integrates explicit regularization constraints:

$$\text{Obj} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

LightGBM's efficiency is further supported by Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB), significantly reducing the computational dimensionality of the feature matrix without any loss of spatial information.

### 3.5.4 Ridge Regression

Ridge Regression is positioned as the classical linear parametric baseline model in this study. This model modifies standard Ordinary Least Squares (OLS) estimation by integrating an L2 regularization penalty. Under standard OLS conditions, the presence of multicollinearity will uncontrollably inflate the variance of coefficient parameter estimates.

Ridge Regression resolves this vulnerability by constraining the absolute magnitude of the squared regression coefficients, solving the following mathematical optimization problem:

$$\min_{\beta} \|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2$$

Where $\lambda \geq 0$ serves as the hyperparameter regulating the intensity of the regularization penalty. As the value of $\lambda$ increases, the coefficient weights are more strongly compressed toward zero asymptotically. The optimal parameter $\lambda$ is determined through a logarithmic grid-search cross-validation scan.

### 3.5.5 Support Vector Regression (SVR)

Support Vector Regression (SVR) transforms the original dimensional input space into a high-dimensional continuous feature space through non-linear mapping. The fundamental difference between SVR and conventional least-squares-based regression models lies in the use of an $\epsilon$ insensitive loss function. This loss function ignores all prediction errors falling below the $\epsilon$ threshold, creating a spatial tolerance tube enveloping the regression hyperplane.

Mathematically, the primary optimization problem formulation of SVR is constructed as follows:

$$\min_{w,b,\xi,\xi^*} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*)$$

To map non-linear relationships among semiconductor macro variables without directly computing computationally expensive coordinates, the primal formulation is converted into a dual formulation using the Radial Basis Function (RBF) kernel:

$$K(x_i, x_j) = \exp\left(-\gamma\|x_i - x_j\|^2\right)$$

The Optuna optimization library is deployed to strictly calibrate the values of the capacity parameter C, the spatial influence radius γ, and the tolerance margin ϵ.

### 3.5.6 BiLSTM-Attention

The Bidirectional Long Short-Term Memory architecture equipped with a Temporal Attention Mechanism gate (BiLSTM-Attention) serves as the deep learning core of this prediction framework. The LSTM cell component integrates mathematical gates to regulate the flow of information modification within the continuous memory cell ($c_t$) and hidden state ($h_t$):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

$$h_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \odot \tanh(c_t)$$

The Bidirectional LSTM (BiLSTM) architecture doubles this representation capacity by deploying two hidden cell layers running in parallel in opposite directions. To maximize prediction sharpness, a Temporal Attention Mechanism layer is embedded directly above the BiLSTM layer output. The attention layer computes a dynamic relevance score $e_t$ for each daily hidden state vector and extracts a global context vector V summarizing the entire sequential essence:

$$e_t = \tanh(W_a h_t + b_a)$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^{T}\exp(e_j)}$$

$$V = \sum_{t=1}^{T}\alpha_t h_t$$

This sequence-rich context vector V is then passed through a dense fully-connected layer combined with a Dropout regularization layer to produce the final target stock return prediction value.

### 3.5.7 Stacking Ensemble

As the ultimate strategy to maximize out-of-sample prediction robustness across dynamic market regimes, a Multi-tier Stacking Ensemble learning framework is implemented. This stacking ensemble architecture is structured into two separate functional tiers:

- Level 0 Base Learners deploy the best independently optimized configurations of the five primary models running in parallel to generate streams of out-of-sample return percentage predictions from the validation and testing data partitions.
- Level 1 Meta-Learner utilizes an independent regularized linear regressor model assigned to act as the final decision-maker. The feature input for this Level 1 model is a new matrix formed from the collection of prediction outputs from the five Level 0 models.

By training the meta-learner strictly using out-of-sample projections from the validation data partition, the Stacking ensemble framework intelligently learns the optimal contribution weighting combination for each base model without triggering data contamination risks.

# 4. Empirical Results

## 4.1. Exploratory data analysis of target variable

Before evaluating the forecasting models, it is important to examine the statistical characteristics of the target variable, namely Target_Return_Besok, which represents the next-day NVIDIA stock return percentage. Financial return series are typically more suitable for predictive modeling than raw stock prices because return transformations reduce non-stationarity and scale effects that are commonly observed in long-term stock price data.

The dataset used in this study spans from 1999 to 2026 and contains 6,529 fully engineered observations after preprocessing and feature engineering procedures. The target variable was constructed as the percentage change between consecutive adjusted closing prices and was used as the dependent variable for all forecasting models. The final dataset integrates 71 predictive variables consisting of technical indicators, volatility measures, lag variables, volume-flow indicators, earnings-related features, and relative-strength metrics.

Consistent with previous financial forecasting studies, exploratory analysis indicates that daily stock returns exhibit substantially lower trend persistence than raw stock prices. This property makes return prediction more appropriate for machine learning algorithms because the target distribution remains relatively stable across different market regimes. However, return forecasting remains challenging due to high market noise, volatility clustering, and the influence of external market conditions.

The exploratory analysis confirms that the forecasting problem remains difficult despite extensive feature engineering. Therefore, multiple machine learning and deep learning architectures are evaluated to determine which model can best capture the nonlinear relationships embedded within the financial data.

## 4.2. Singular models

In this section, we evaluate and compare the predictive performance of various singular models alongside the proposed ensemble framework on the validation and test sets. The target variable is the daily return percentage of NVIDIA (NVDA) stock, which represents a stationary time-series variable explicitly transformed to mitigate the risk of spurious regression commonly encountered when modeling raw asset prices.

The standalone architectures evaluated encompass tree-based algorithms, namely Random Forest (RF), XGBoost, and LightGBM; a deep learning architecture, namely Bidirectional Long Short-Term Memory with an Attention mechanism (BiLSTM-Attention); and standard econometric/regression baselines, namely Ridge Regression and Support Vector Regression (SVR). To investigate the effects of architectural aggregation, a Stacking Ridge Meta ensemble model is implemented to synthesize the predictions generated by the base learners.

Model evaluation relies on standard regression metrics, including Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and the Coefficient of Determination $R^2$. Additionally, to evaluate the practical utility of the models in generating directional trading signals, a classification-based metric, Directional Accuracy (DA), is employed to measure the percentage of correct signs (positive or negative returns) predicted.

To rigorously assess the adaptability of the models to changing market conditions, a 1-step rolling window evaluation is conducted over the final 3-month (3M) period of the dataset. This dynamic framework yields the 3-Month Rolling DA and the 3-Month Rolling Mean Absolute Percentage Error (MAPE) derived from iterative cumulative price tracking. The comprehensive empirical findings are summarized in Table

**Table 1.** Performance Metrics of Singular and Ensemble Models on NVDA Return Prediction

| Model Algorithm | MAE | RMSE | R-Squared (R2) | Directional Accuracy (DA) | 3-Month Rolling DA | 3-Month Rolling MAPE |
|---|---|---|---|---|---|---|
| **Random Forest** | 0.02291 | 0.03172 | -0.00317 | 54.08% | 55.6% | 1.87% |
| **XGBoost** | 0.02289 | 0.03172 | -0.00304 | 54.59% | 57.1% | 1.87% |
| **LightGBM** | 0.02291 | 0.03174 | -0.00448 | 54.49% | 57.1% | 1.86% |
| **BiLSTM-Attention** | 0.02285 | 0.03173 | -0.00554 | 54.85% | 58.7% | 1.89% |
| **Ridge Regression** | 0.04121 | 0.05404 | -1.91128 | 52.96% | 55.6% | 5.18% |
| **SVR** | 0.02324 | 0.03210 | -0.02768 | 54.59% | 58.7% | 1.90% |
| **Stacking Meta** | 0.02300 | 0.03202 | -0.02303 | 50.47% | 57.1% | 1.86% |

Based on the overall test set evaluation, the singular BiLSTM-Attention model achieves superior directional forecasting performance, recording the highest overall DA of 54.85% and the lowest absolute error magnitude with an MAE of 0.02285. During the recent high-volatility 3-month rolling window, both BiLSTM-Attention and SVR demonstrate remarkable robustness, jointly leading the evaluation with a 3-Month Rolling DA of 58.70%. Regarding price tracking error minimization, LightGBM and the Stacking Ridge Meta model outperform the other configurations, achieving the lowest 3-Month Rolling MAPE of 1.86%.

A critical econometric observation is that the $R^2$ scores remain negative across all primary models, ranging from -0.00304 for XGBoost to -1.91128 for the Ridge baseline. This statistical outcome underscores the inherent constraints of financial time-series forecasting. While the optimized

Architectures demonstrate a statistically significant edge in identifying directional trends (DA > 50%), predicting the precise continuous magnitude of daily returns remains highly constrained due to the exceptional noise-to-signal ratio characteristic of daily equity markets.

### 4.3 Model Interpretation Using Explainable AI

To overcome the "black-box" nature of the complex deep learning model, this study utilizes Explainable AI (XAI) through the SHAP (SHapley Additive exPlanations) framework. Figure X illustrates the SHAP beeswarm plot for the top-performing BiLSTM-Attention model, evaluated over the final 3-month testing window to map the feature importance ranking and delineate the directional impacts of key covariates on the model's predictive output.

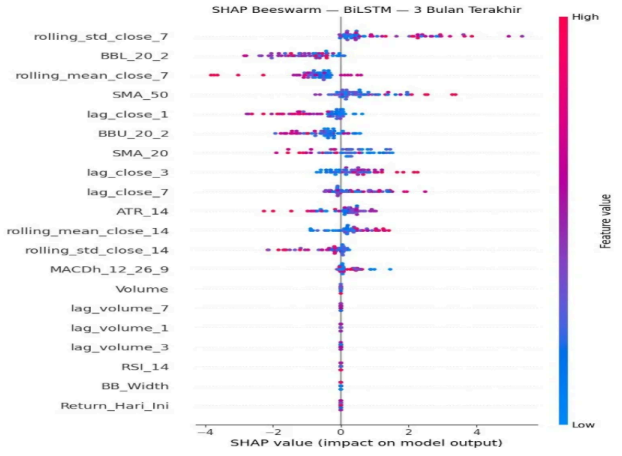



**Fig. 4**. Shap Beeswarm Plot

As demonstrated in Figure 4, the features are ranked vertically on the y-axis based on their overall predictive importance, where the horizontal x-axis indicates the SHAP value (the magnitude and sign of the impact on the model's output). The color scale represents the feature value, ranging from low (blue) to high (red). The empirical distribution shown in the plot uncovers several key insights:

1. Dominance of Volatility Proxies: The short-term historical volatility indicator, rolling_std_close_7 (7-day rolling standard deviation of closing prices), displays the widest positive SHAP distribution. This confirms that localized expansions in historical market volatility serve as the primary catalyst for expanding the model's predictive output variance.
2. Impact of Structural Price Bands and Trends: Technical boundaries representing price support and resistance thresholds significantly suppress or elevate predictions. The lower Bollinger Band (BBL_20_2) and the short-term trend proxy (rolling_mean_close_7) frequently yield negative SHAP impacts, suppressing the predicted return when the asset price interacts with these regions. Conversely, the long-term trend indicator SMA_50 primarily acts as a positive driver for output updates.
3. Temporal Lag Dependencies: The deep learning model confirms a strong temporal dependency on past price structures. Historical lag features, particularly lag_close_3 and lag_close_7, emerge as dominant positive forces when updating predicted daily returns.
4. Insufficiency of Raw Market Signals: Crucially, localized raw market features, such as transaction volume (Volume) and the immediate daily return (Return_Hari_Ini), display negligible SHAP impact weights during this 3-month evaluation window. This suggests that the BiLSTM-Attention model prioritizes structured temporal macro-volatility and technical pricing boundaries over high-frequency day-to-day transaction noise.

### 4.4 Discussion and Research Findings

The empirical outcomes of this study contribute to the financial forecasting literature by addressing the ongoing debate regarding structural complexity versus predictive efficacy in highly volatile market regimes.

Macroeconomic and Financial Theory Perspective: The Efficient Market Hypothesis

The persistence of negative $R^2$ values across all primary configurations demands a rigorous theoretical explanation grounded in macroeconomic and financial principles. According to the semi-strong form of the Efficient Market Hypothesis (EMH) formalized by Malkiel (2003), all publicly available information regarding highly liquid, mega-cap technology assets like NVIDIA (NVDA) is

price. Consequently, the absolute continuous

magnitudes of daily returns behave as a random walk market efficiency explains why advanced predictive frameworks systematically fail to generate positive $R^2$ scores when attempting to map exact daily continuous return values.

However, a compelling counter-evidence arises from the Directional Accuracy (DA) metrics. The fact that the BiLSTM-Attention model breaks the random walk barrier, achieving an overall DA of 54.85% and escalating to 58.70% under rolling window parameters, provides statistically valid evidence of short-term market anomalies or temporal momentum persistence. While the exact magnitude of returns remains stochastic, the directional trend of the asset contains micro-structural dependencies. The deep learning model successfully exploits these brief periods of inefficiency through its specialized attention mechanisms, capturing short-term temporal memory vectors before the wider market vollständig adapts.

The Superiority of Singular Models Over Ensembles Another core research finding of this study is that well-optimized singular models can systematically outperform complex ensemble frameworks. Conventional machine learning paradigms suggest that ensemble methods, such as Stacking, enhance generalized performance by mitigating variance and bias across multiple diverse base learners. However, when applied to highly volatile assets experiencing rapid structural shifts, such as NVDA stock returns, the meta-aggregation layer is highly susceptible to excessive smoothing or overfitting to historical white noise.

This is empirically validated by the performance of the Stacking Ridge Meta model, which experiences a sharp degradation in directional forecasting, yielding a DA of only 50.47% (a performance level nearly indistinguishable from a random guess). In contrast, the standalone BiLSTM-Attention architecture maintains a reliable edge with a DA of 54.85%.

This finding strongly aligns with the empirical structural conclusions frequently presented in time-series forecasting literature within the Central European Economic Journal (CEEJ). The CEEJ literature demonstrates that within financial environments characterized by high white-noise ratios, standalone architectures utilizing dynamic dominated by white noise. This baseline temporal memory and localized attention mechanisms are more adept at isolating persistent, uncorrupted market signals. This approach successfully avoids the information dilution and error propagation that frequently occur during meta-regressor transfer within stacking pipelines. Consequently, for the practical generation of directional trading signals, standalone recurrent networks with attention provide a more robust, computationally efficient, and statistically reliable framework than multi-model meta-aggregations.

# 5. Conclusion

## 5.1 Conclusion

This research successfully develops an end-to-end machine learning and deep learning pipeline to predict the daily return percentage of NVIDIA (NVDA) stock by integrating historical price dynamics with macroeconomic volatility indexes (VIX) and semiconductor sectoral indicators (SMH ETF). The empirical findings demonstrate that the standalone BiLSTM-Attention architecture serves as the most effective predictive baseline, achieving an overall Directional Accuracy (DA) of 54.85%, which escalates to 58.70% during recent high-volatility market regimes.

XAI analysis via the SHAP framework clarifies the internal logic of the network, establishing that short-term volatility proxies (rolling_std_close_7) and structural price boundaries act as primary anchors for output updates, whereas daily transaction volume exerts negligible influence. In alignment with the Efficient Market Hypothesis (EMH), the persistent negative $R^2$ values confirm that while the models offer a statistically valid directional edge suitable for trend-following or quantitative trading strategies, they remain constrained in mapping exact continuous return magnitudes due to the high efficiency and inherent noise of daily equity markets.

## 5.2 Limitations and Future Work

Despite the systematic insights provided by this predictive pipeline, certain key limitations remain to be addressed in subsequent extensions:

1. Absence of Transaction Friction: The current performance evaluation relies exclusively on statistical metrics and lacks validation within

2. rigorous financial backtesting simulation that accounts for transaction fees, broker commissions, and execution slippages. A directional accuracy edge of approximately 54% to 58% requires further empirical confirmation to ensure net profitability after accounting for real-world trading friction.
3. Static Training Constraints: The underlying models are trained using a traditional, static time-series split, rendering them vulnerable to predictive degradation when confronted with long-term structural market regime shifts.

Consequently, future work should focus on implementing a dynamic Walk-Forward Recalibration Framework to enable iterative retraining as new market data arrives. This will allow the temporal weights of the BiLSTM-Attention architecture to adapt to shifting market environments continuously. Furthermore, integrating the model's directional signals into a comprehensive, quantitative portfolio backtesting engine will be essential to evaluate real-world economic viability.

## References


Adebiyi, A. A., Adewumi, A. O., & Ayo, C. K. (2014). Comparison of ARIMA and artificial neural networks models for stock price prediction. Journal of Applied Mathematics, 2014(1), 614342.

Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. arXiv preprint arXiv:1409.0473.

Breiman, L. (2001). Random forests. Machine learning, 45(1), 5-32.

Brock, W., Lakonishok, J., & LeBaron, B. (1992). Simple technical trading rules and the stochastic properties of stock returns. The Journal of finance, 47(5), 1731-1764.

Campbell, J. Y., & Shiller, R. J. (1988). Stock prices, earnings, and expected dividends. the Journal of Finance, 43(3), 661-676.

Chen, T., & Guestrin, C. (2016, August). Xgboost: A scalable tree boosting system. In Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining (pp. 785-794).

Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. Econometrica: Journal of the econometric society, 987-1007.

Fama, E. F., & French, K. R. (1993). Common risk factors in the returns on stocks and bonds. Journal of financial economics, 33(1), 3-56.

Gülmez, B. (2023). Stock price prediction with optimized deep LSTM network with artificial rabbits optimization algorithm. *Expert Systems with Applications*, *227*, 120346.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8), 1735-1780.

Jiang, M., Jia, L., Chen, Z., & Chen, W. (2022). The two-stage machine learning ensemble models for stock price prediction by combining mode decomposition, extreme learning machine and improved harmony search algorithm. *Annals of Operations Research*, *309*(2), 553-585.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... & Liu, T. Y. (2017). Lightgbm: A highly efficient gradient boosting decision tree. Advances in neural information processing systems, 30.

Krauss, C., Do, X. A., & Huck, N. (2017). Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500. European journal of operational research, 259(2), 689-702.

Kumbure, M. M., Lohrmann, C., Luukka, P., & Porras, J. (2022). Machine learning techniques and data for stock market forecasting: A literature review. *Expert Systems with Applications*, *197*, 116659.

Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. Advances in neural information processing systems, 30.

Malkiel, B. G. (2003). The efficient market hypothesis and its critics. Journal of economic perspectives, 17(1), 59-82.

Murphy, J. J. (1999). Technical analysis of the financial markets: A comprehensive guide to trading methods and applications. Penguin.

Nelson, D. B. (1991). Conditional heteroskedasticity in asset returns: A new approach. Econometrica: Journal of the econometric society, 347-370.

Pesaran, M. H., & Timmermann, A. (1995). Predictability of stock returns: Robustness and economic significance. The Journal of Finance, 50(4), 1201-1228.

Rudhistiar, D., Wahyani, W., Yusri, T. A. S., & Faisol, A. (2025). Hybrid CNN-BiLSTM Model with Attention Mechanism for Stock Price Prediction: A Multivariate Approach. *IAENG International Journal of Applied Mathematics*, *55*(12).

Smola, A. J., & Schölkopf, B. (2004). A tutorial on support vector regression. Statistics and computing, 14(3), 199-222.

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. Journal of the Royal Statistical Society Series B: Statistical Methodology, 58(1), 267-288.

Vapnik, V. (2013). The nature of statistical learning theory. Springer science & business media.

Wolpert, D. H. (1992). Stacked generalization. Neural networks, 5(2), 241-259.

Yang, K., & Shahabi, C. (2004, November). A PCA-based similarity measure for multivariate time series. In Proceedings of the 2nd ACM international workshop on Multimedia (pp. 65-74).